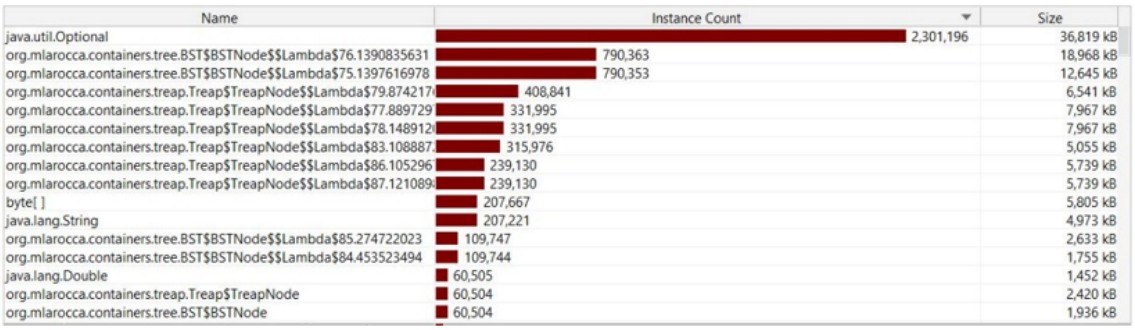
Try to get a balance between clean code, time used to develop it and efficiency: “premature optimization is the root of all evil”, as stated by Donald Knuth[16], because trying to optimize all of your code will likely distract your team from finding the critical issues, and produce less clean, less readable and less maintainable code.

So, always make sure to try to write clean code first, and then optimize first the bottlenecks and the critical sections, for which you have a service level agreement you’re your clients or anyway you have requirements about time/memory used.

To give you a concrete example, the Java implementation we provide on the book’s repo makes large use of the `Optional` class (to avoid using null, and provide a nicer interface and a nicer way to handle unsuccessful searches/operations), and consequently also a lot of lambda functions.

If we profile more in detail the memory usage, disabling the filter on the package (to speed-up profiling, you usually want to avoid recording standard libraries etc…), the final result is quite surprising, as we can see in figure 3.25.



Name	Instance Count	Size
java.util.Optional	2,301,196	36,819 kB
org.mlarocca.containers.tree.BST$BSTNode$$Lambda$76.1390835631	790,363	18,968 kB
org.mlarocca.containers.tree.BST$BSTNode$$Lambda$75.1397616978	790,353	12,645 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$79.874217	408,841	6,541 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$77.889729	331,995	7,967 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$78.148912	331,995	7,967 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$83.108887	315,976	5,055 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$86.105296	239,130	5,739 kB
org.mlarocca.containers.treap.Treap$TreapNode$$Lambda$87.121089	239,130	5,739 kB
byte[ ]	207,667	5,805 kB
java.lang.String	207,221	4,973 kB
org.mlarocca.containers.tree.BST$BSTNode$$Lambda$85.274722023	109,747	2,633 kB
org.mlarocca.containers.tree.BST$BSTNode$$Lambda$84.453523494	109,744	1,755 kB
java.lang.Double	60,505	1,452 kB
org.mlarocca.containers.treap.Treap$TreapNode	60,504	2,420 kB
org.mlarocca.containers.tree.BST$BSTNode	60,504	1,936 kB

Figure 3.25 Memory allocation by class for our test, without any filtering on the package.

You can notice that most space is used by instances of `Optional` and lambdas (implicitly created in `Optional::map` etc…).